

Day 11 复习总表：异步编程基础（下）——fetch API 与 async/await

Step 项	完成事项	对应代码 / 指令	代码详解
Step 2 fetch() 基础用法	用 fetch 请求真实公开 API 数据	<pre>js fetch(url).then(r => r.json()).then(data => {...}).catch(e => {...});</pre>	• <code>fetch()</code> 返回 Promise，第一个 <code>.then()</code> 拿到 <code>response</code>（响应外壳） • <code>response.json()</code> 本身也是异步的，第二个 <code>.then()</code> 才拿到真正的 <code>data</code> • <code>.catch()</code> 捕获请求链条中的任何错误
Step 3 async/await 语法	用更简洁的写法改写 Promise 链	<pre>js async function getUserData() { const response = await fetch(url); const data = await response.json(); }</pre>	• <code>async</code> 标记函数为异步函数，内部才能用 <code>await</code> • <code>await</code> 让代码"看起来暂停"，直到 Promise resolve • 本质是 Promise 的语法糖，底层机制不变，只是更接近同步代码的阅读体验
Step 4 try/catch 错误处理	优雅处理请求失败的情况	<pre>js try { if (!response.ok) throw new Error(...); ... } catch (error) { ... } finally { ... }</pre>	• <code>fetch</code> 只在网络层失败（断网等）才自动 reject；HTTP 错误状态码（404/500）仍会 resolve，必须手动检查 <code>response.ok</code> 并 <code>throw</code> • <code>catch</code> 只在出错时执行；<code>finally</code> 无论成功失败都必定执行一次
Step 5 综合练习：渲染真实数据	把 fetch 到的数据用 map 渲染成 HTML（选做）	<pre>js const posts = await response.json(); posts.map(post => `<article>...\${post.title}... </article>`+`)+`.join('');</pre>	• 数据来源从 Day 8 的本地数组变成网络请求，但 <code>map()</code>+<code>.join('')</code> 转换 HTML 的核心逻辑完全一致 • 这是"数据驱动渲染"思想的自然延伸：数据从哪来不重要，转换逻辑不用变
Step 6 Git 协作：pull + push	练习标准的"先拉取再推送"流程	<pre>bash git pull git add script.js git commit -m "..."</pre> <pre>git push</pre>	• <code>pull</code> = <code>fetch</code>（下载远程更新）+ <code>merge</code>（合并到本地） • 先 <code>pull</code> 再 <code>push</code> 是标准协作流程，避免因远程有新内容而被拒绝推送或产生冲突

Step 项	完成事项	对应代码 / 指令	代码详解
Step 7 Code Review + 自评	6 道概念题检验理解	考察问题： 1. response vs data 2. async/await 本质 3. fetch 为何不自动捕获 HTTP 错误 4. finally vs catch 5. 数据驱动渲染的一致性 6. 为何先 pull 再 push	• response=响应外壳，data=解析后的真实数据 • async/await 是 Promise 的语法糖 • fetch 只在网络层失败时 reject，HTTP 错误需手动判断 <code>response.ok</code> • finally 无条件执行，catch 只在 reject 时执行 • map+join 的转换逻辑与数据来源无关 • 先 pull 避免推送被拒或产生合并冲突

本日核心概念速查

概念	一句话记忆
<code>response</code> vs <code>data</code>	前者是响应外壳，后者是解析后的真数据，中间隔着一层 <code>.json()</code>
async/await	Promise 的语法糖，让异步代码读起来像同步代码
<code>response.ok</code>	fetch 不会因 404/500 自动报错，必须手动检查这个属性
<code>finally</code>	唯一"无论如何都执行"的代码块，常用于收尾
<code>pull</code> → <code>push</code>	协作的标准顺序：先同步远程最新内容，再推送自己的改动